



A population-based iterated greedy algorithm for the minimum weight vertex cover problem[☆]

Salim Bouamama^{a,c}, Christian Blum^{b,*}, Abdellah Boukerram^c

^a Department of Computer Science, University of M'sila, M'sila 28000, Algeria

^b ALBCOM Research Group, Universitat Politècnica de Catalunya, Barcelona, Spain

^c Department of Computer Science, Faculty of Sciences, Ferhat Abbas University, Sétif 19000, Algeria

ARTICLE INFO

Article history:

Received 12 July 2011

Received in revised form 13 February 2012

Accepted 19 February 2012

Available online 3 March 2012

Keywords:

Iterated greedy algorithm

Minimum weight vertex cover problem

Population-based techniques

ABSTRACT

Given an undirected, vertex-weighted graph, the goal of the minimum weight vertex cover problem is to find a subset of the vertices of the graph such that the subset is a vertex cover and the sum of the weights of its vertices is minimal. This problem is known to be NP-hard and no efficient algorithm is known to solve it to optimality. Therefore, most existing techniques are based on heuristics for providing approximate solutions in a reasonable computation time.

Population-based search approaches have shown to be effective for solving a multitude of combinatorial optimization problems. Their advantage can be identified as their ability to find areas of the space containing high quality solutions. This paper proposes a simple and efficient population-based iterated greedy algorithm for tackling the minimum weight vertex cover problem. At each iteration, a population of solutions is established and refined using a fast randomized iterated greedy heuristic based on successive phases of destruction and reconstruction. An extensive experimental evaluation on a commonly used set of benchmark instances shows that our algorithm outperforms current state-of-the-art approaches.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

The minimum weight vertex cover (MWVC) problem is a fundamental graph problem with many important real-life applications such as, for example, in wireless communication, circuit design and network flows. Given an undirected graph $G=(V,E)$ with weights on its vertices, the goal of the minimum weight vertex cover problem is to find among all subsets $S \subset V$ that are vertex covers a subset S^* for which the sum of the weights of the vertices is minimal. The decision version of the problem is NP-complete [1]. Due to its inherent computational complexity, most developed techniques are based on heuristics for providing approximate solutions in reasonable computation time. In fact, most of the developed algorithms are greedy heuristics. The first one was introduced in [2]. It starts with an empty partial solution and adds one vertex at a time. More specifically, at each step it chooses the vertex with the minimum ratio between vertex weight and current degree. Note that the *current degree* is only determined by edges that are not yet covered.

Pitt [3] proposed a randomized 2-approximation algorithm which repeatedly selects one end-point (vertex) of an arbitrarily chosen edge with a probability inversely proportional to its weight. A survey of approximation algorithms for the minimum weight vertex cover problem can be found in [4]. From complexity theory it is known that – unless $P=NP$ – it is impossible to approximate the vertex cover problem, which is a special case of the MWVC problem in which all vertices are of unit weight, within any factor smaller than $10\sqrt{5} - 21 \approx 1.3606$ [5]. Moreover, it might be hard to approximate it within an approximation ratio of $2 - \varepsilon$ for all $\varepsilon > 0$ [6].

In addition to heuristics, several metaheuristics have been devised for tackling the MWVC problem. Khuri and Bäck [7] treated MWVC as a constrained combinatorial optimization problem and introduced a genetic algorithm that makes use of a graded penalty term incorporated in the fitness function to penalize infeasible solutions. Furthermore, a variant of a genetic algorithm hybridized with a greedy heuristic has been reported in [8]. Recently, an ant colony optimization (ACO) was proposed by Shyu et al. in [9]. According to their empirical results ACO outperforms others metaheuristics such as tabu search and simulated annealing. This algorithm was further improved in [10] by incorporating a pheromone correction strategy in the hope to avoid stagnation of the search and the convergence to local optima. Finally, a metaheuristic based on gravity has been described in [11]. The results of this approach were compared to genetic algorithms and some greedy heuristics. However, in general the proposed metaheuristics seem not to scale well. Moreover,

[☆] This work was supported by grant TIN2007-66523 (FORMALISM) of the Spanish Government. In addition, Christian Blum acknowledges support from the *Ramón y Cajal* program of the Spanish Government of which he is a research fellow.

* Corresponding author.

E-mail addresses: bouamamas@gmail.com (S. Bouamama), cblum@lsi.upc.edu (C. Blum), aboukerram@univ-setif.dz (A. Boukerram).

they seem to be quite time consuming for large size problem instances.

1.1. Contributions of this work

In this work, we propose a population-based iterated greedy (PBIG) algorithm which is able to provide state-of-the-art results in a short computational time for the MWVC problem. Our approach belongs to the class of iterated greedy (IG) algorithms that have recently received considerable attention regarding their potential for effectively solving a wide range of difficult optimization problems, including the founder sequence reconstruction problem [12], various scheduling problems [13–15] and the maximum diversity problem [16]. At each iteration, IG algorithms first partially destroy the incumbent solution before a greedy heuristic is used to derive a complete solution from the partially destroyed solution [17]. According to Hoos and Stützle [17], two important advantages of starting the solution construction from partial solutions are that (i) the solution construction process is much faster and (ii) good parts of solutions may be exploited directly.

For the construction phase of our PBIG algorithm, we develop a randomized greedy construction heuristic based on the heuristic described in [2]. During the destruction phase, a fixed number of randomly chosen vertices (generally referred to as solution components) are removed from the incumbent solution. Hereby, the size of destruction is determined as a fixed proportion of the incumbent solution. Another contribution of our work is the extension of the IG algorithm to be able to work on populations of solutions, rather than being restricted to a single incumbent solution. The experimental evaluation shows that this extension contributes significantly to the quality of the obtained results. We assess the performance of our algorithm on all benchmark instances that have been considered by Shyu et al. in [9] and Jovanovic and Tuba in [10]. A part of these benchmark sets have also been used in [11]. We also conduct extensive tuning experiments for finding appropriate parameter settings for the proposed algorithm. The obtained results show that PBIG is able to outperform recent state-of-the-art algorithms such as the ones proposed in [9–11]. Another advantage of our approach is the fact that it is able to derive good solutions for difficult problem instances in a short computation time.

1.2. Organization

The organization of this work is as follows. A technical description of the MWVC problem is presented in Section 2. The proposed population-based iterated greedy algorithm is described in Section 3. Finally, an extensive experimental evaluation is given in Section 4, while Section 5 offers conclusions and an outlook to future work.

2. The minimum weight vertex cover problem

A problem instances (G, ω) of the MWVC problem is a tuple that consists of an undirected graph $G(V, E)$, where V is the set of vertex and E is the set of edges, and a function $\omega : V \rightarrow \mathbb{R}^+$ that associates a positive weight value $\omega(v)$ to each vertex $v \in V$. The MWVC problem can be formally defined as follows:

$$\begin{aligned} \text{minimize } \omega(S) &:= \sum_{v \in S} \omega(v) \\ \text{subject to } \forall (v_i, v_j) \in E : (v_i \in S) \text{ or } (v_j \in S), \\ &S \subseteq V. \end{aligned}$$

In the above definition, a *candidate solution* to the problem, denoted by S , is a subset of V . A candidate solution S is a valid solution, a so-called *vertex cover*, if each edge in G has at least one endpoint

in S . The objective function value $\omega(S)$ of a candidate solution S is defined as the sum of the weights of the vertices in S . The objective of the MWVC problem is to find a valid candidate solution that minimizes the objective function.

Throughout the paper, we denote by $n = |V| = \{v_1, v_2, \dots, v_n\}$ the number of vertices and by $m = |E|$ the number of edges in G . Moreover, we assume that G is given by means of an adjacency matrix $A = \{a_{ij} \mid i = 1, \dots, n; j = 1, \dots, n\}$ where

$$a_{ij} = \begin{cases} 1 & \text{if } \exists (v_i, v_j) \in E, \\ 0 & \text{otherwise} \end{cases}$$

Finally, we denote by $d(v)$ the degree of a vertex $v \in V$, that is, the number of edges incident to this vertex, which is defined as

$$d(v) := \sum_{j=1}^n a_{vj} = \sum_{j=1}^n a_{ji}$$

3. The proposed algorithm

In the following we outline the PBIG algorithm that we developed for the MWVC problem. As mentioned before, PBIG is a population-based iterated greedy algorithm where a population of solutions, starting from an initial population, is iteratively improved by means of alternating between destruction and reconstruction until termination conditions are met.

Constructive methods are characterized by the fact that they work on partial solutions. Each constructive method is based on a mechanism for extending partial solutions. Generally, a partial solution S may be extended by adding a solution component from a finite set $\mathcal{N}(S)$ of solution components that are allowed to be added. In the case of the MWVC problem, $\mathcal{N}(S) \subseteq V \setminus S$ represents the set of vertices that have at least one incident edge which is still uncovered, that is, an edge of which both endpoints are not in S . Given a (partial) solution S , we henceforth refer to $d_S(v_i) := |\{(v_i, v_j) \in E \mid v_j \notin S\}|$ as the *current degree* of a vertex $v \in V \setminus S$ with respect to S . With this definition $\mathcal{N}(S)$ can technically be defined in the following way:

$$\mathcal{N}(S) := \{v \in V \setminus S \mid d_S(v) \neq 0\} \quad (1)$$

Note that when S is a complete solution it holds that $\mathcal{N}(S) = \emptyset$. On the other hand, at the start of the algorithm, when S is empty, it holds that $\mathcal{N}(S) = \mathcal{N}_0$ where $\mathcal{N}_0 = \{v \in V \mid d(v) \neq 0\}$, i.e. the isolated vertices are excluded.

A high level description of our algorithm is given in Algorithm 1. Apart from a problem instance (G, ω) , PBIG requires three input parameters to control its flow: (1) the population size $pop_size \in \mathbb{Z}^+$, (2) the degree of destruction $p \in [0, 1] \subset \mathbb{R}$, which is a parameter that is used to determine the size of the destruction, and (3) the determinism rate $\alpha \in [0, 1] \subset \mathbb{R}$. The latter parameter is used to control the degree of stochasticity of the solution construction procedure.

The algorithm works as follows. First, the pop_size solutions of the initial population are generated by function `GenerateInitialPopulation(pop_size)`. Afterwards, each algorithm iteration consists of the following steps. First, an empty population $\mathcal{P}$ called offspring population is created. Then, each solution $S_k \in \mathcal{P}$ ($k = 1, \dots, pop_size$) is partially destroyed using procedure `DestroyPartially(S_k, p)`, resulting in a partial solution S_k^p . On the basis of S_k^p , a complete solution S_k' is then constructed using procedure `GreedyMWVC(S_k^p, \alpha)`. Each newly obtained complete solution is stored in $\mathcal{P}$. Note that the two phases of destruction and construction are applied to all solutions from $\mathcal{P}$ independently of each other. When the iteration is completed, procedure `Accept(\mathcal{P})`,

$\mathcal{P}^p$) selects the best pop_size solutions from $\mathcal{P} \cup \mathcal{P}^p$ for the population of the next iteration. This mechanism biases the search process towards the best solutions found during the search, and keeps the population size fixed to pop_size solutions at all times. Finally, the algorithm terminates when a maximum number of iterations has been performed, and the best found solution is returned.

The three procedures `GenerateInitialPopulation(pop_size)` (see Algorithm 2), `GreedyMWVC(·, α)` (see Algorithm 3) and `DestroyPartially(Sk, p)` that form the core of PBIG are described in more detail in the following.

Algorithm 1. PBIG for the MWVC problem

```

1: input: A problem instance  $(G, \omega)$ , and parameters
    $pop\_size \in \mathbb{Z}^+$ ,  $\alpha \in [0, 1] \subset \mathbb{R}$ ,  $p \in [0, 1] \subset \mathbb{R}$ .
2:  $\mathcal{P} \leftarrow \text{GenerateInitialPopulation}(pop\_size)$  // see
   Algorithm 2
3: while termination condition not satisfied do
4:    $\mathcal{P}^p \leftarrow \emptyset$ 
5:   for each candidate solution  $S_k \in \mathcal{P}$  do
6:      $S_k^p \leftarrow \text{DestroyPartially}(S_k, p)$ 
7:      $S_k' \leftarrow \text{GreedyMWVC}(S_k^p, \alpha)$  // see Algorithm 3
8:      $\mathcal{P}^p \leftarrow \mathcal{P}^p \cup \{S_k'\}$ 
9:   end for
10:   $\mathcal{P} \leftarrow \text{Accept}(\mathcal{P}, \mathcal{P}^p)$ 
11: end while
12: output:  $\text{argmin} \{ \omega(S_k) \mid S_k \in \mathcal{P}, k = 1, \dots, pop\_size \}$ 

```

`GenerateInitialPopulation(pop_size)`: This function generates pop_size solutions for the initial population. For the construction of a solution it applies procedure `GreedyMWVC(·, α)` to the empty partial solution $S_0 = \emptyset$. Since this procedure typically includes an element of stochasticity (depending on parameter α), the generated solutions are generally different from each other. The pseudo-code for generating the initial population is shown in Algorithm 2.

Algorithm 2. Procedure `GenerateInitialPopulation(pop_size)`

```

1: input:  $pop\_size$ 
2:  $\mathcal{P} \leftarrow \emptyset$ 
3:  $S_0 \leftarrow \emptyset$ 
4: for  $k = 1, \dots, pop\_size$  do
5:    $S_k \leftarrow \text{GreedyMWVC}(S_0, \alpha)$  // see Algorithm 3
6:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{S_k\}$ 
7: end for
8: output:  $\mathcal{P} = \{S_1, S_2, \dots, S_{pop\_size}\}$ 

```

`GreedyMWVC(S, α)`: This procedure takes a partial solution S (which might be empty) as input. At each construction step, the current partial solution S is extended by adding one solution component from $\mathcal{N}(S)$ until a complete solution – that is, a vertex cover – is constructed. The choice of the next solution component to be added to S at each step is done in the following way. First, each solution component $v \in \mathcal{N}(S)$ is rated according to a greedy function $cost(\cdot)$ defined as $cost(v) := \omega(v)/d_S(v)^\beta$. When $\beta = 1$ this greedy function is the same as the one used in [2]. However, experimental tests have shown that better results may sometimes be obtained with $\beta = 2$. Therefore, for each application of procedure `GreedyMWVC(S, α)` a value for β is randomly adopted from $\{1, 2\}$.

Let v_1^{best} and v_2^{best} (where $v_1^{best} \neq v_2^{best}$) be the two vertices from $\mathcal{N}(S)$ with lowest cost values. Moreover, let $cost(v_1^{best}) \leq cost(v_2^{best})$. The determinism rate α is used to decide which one of the two solution components is chosen and added to S . This is done by firstly generating a random value α_0 uniformly distributed over $[0, 1]$. If $\alpha_0 \leq \alpha$ then v_1^{best} is chosen, otherwise v_2^{best} is chosen. The adding of vertices to S stops when $\mathcal{N}(S) = \emptyset$.

At this point solution S may contain vertices whose neighbors – that is, adjacent vertices – belong to S as well. Given a solution S this set may be formally defined as follows:

$$S^u := \{v \in S \mid \mathcal{A}(v) \subseteq S\}, \quad (2)$$

where $\mathcal{A}(v)$ is the set of vertices that are adjacent to vertex v in graph G . Note that vertices from S^u do not contribute to the covering and may safely be removed from S . This is done iteratively. While S^u is not empty, one vertex from S^u is chosen as follows. First, each vertex $v \in S^u$ is rated according to a greedy function $rcost(\cdot)$, which is obtained by inverting the greedy function $cost(\cdot)$ from the construction process as explained above. More specifically, $rcost(v) := d(v)^\beta/\omega(v)$. Hereby, parameter β adopts the value that was chosen for the construction phase. This iterative process stops once S^u is empty.

The pseudo-code of the complete function is shown in Algorithm 3.

Algorithm 3. Procedure `GreedyMWVC(S, α)`

```

1: input: a partial solution  $S$ , and parameter  $\alpha$ 
2: Choose randomly a value for  $\beta$  from  $\{1, 2\}$ 
3:  $\mathcal{N}(S) \leftarrow \{v \in V \setminus S \mid d_S(v) \neq 0\}$ 
4: while  $\mathcal{N}(S) \neq \emptyset$  do
5:    $v_1^{best} \leftarrow \text{argmin} \{cost(v) \mid v \in \mathcal{N}(S)\}$ 
6:    $v_2^{best} \leftarrow \text{argmin} \{cost(v) \mid v \in \mathcal{N}(S) \setminus \{v_1^{best}\}\}$ 
7:    $\alpha_0 \leftarrow \text{random number uniformly distributed over } [0, 1]$ 
8:   if  $\alpha_0 \leq \alpha$  then  $v^{best} \leftarrow v_1^{best}$ 
9:   else  $v^{best} \leftarrow v_2^{best}$ 
10:   $S \leftarrow S \cup \{v^{best}\}$ 
11:   $\mathcal{N}(S) \leftarrow \{v \in V \setminus S \mid d_S(v) \neq 0\}$ 
12: end while
13:  $S^u \leftarrow \{v \in S \mid \mathcal{A}(v) \subseteq S\}$ 
14: while  $S^u \neq \emptyset$  do
15:    $v_1^{best} \leftarrow \text{argmax} \{rcost(v) \mid v \in S^u\}$ 
16:    $v_2^{best} \leftarrow \text{argmax} \{rcost(v) \mid v \in S^u \setminus \{v_1^{best}\}\}$ 
17:    $\alpha_0 \leftarrow \text{random number uniformly distributed over } [0, 1]$ 
18:   if  $\alpha_0 \leq \alpha$  then  $v^{best} \leftarrow v_1^{best}$ 
19:   else  $v^{best} \leftarrow v_2^{best}$ 
20:    $S \leftarrow S \setminus \{v^{best}\}$ 
21:    $S^u \leftarrow \{v \in S \mid \mathcal{A}(v) \subseteq S\}$ 
22: end while
23: output: a complete solution  $S$ 

```

`DestroyPartially(S, p)`: the destruction procedure starts with a complete solution S . Then, $\lfloor p \times |S| \rfloor$ randomly chosen vertices – where $0 < p \leq 1$ – are removed from S in an iterative way. Hereby, at each step exactly one randomly chosen vertex is removed. Parameter p is henceforth called the *degree of destruction*.

3.1. Complexity considerations

The run time complexity of the proposed method is dominated by procedure `GreedyMWVC(S, α)` as shown in Algorithm 3. In fact, the complexity of this algorithmic component is the same as the one of the classical greedy heuristic from [2] for set covering and vertex covering. In other words, the complexity is polynomial in the number of vertices and edges of the input graph. The exact run time complexity depends on the data structures that are used to keep and modify the input graph during the execution of the algorithm. Finally, the run time complexity of one iteration of the proposed PBIG algorithms is pop_size times the run time complexity of procedure `GreedyMWVC(S, α)`.

4. Experimental evaluation

The proposed algorithm was implemented in ANSI C++ using GCC 4.4 for compiling the software, and experimentally evaluated on a cluster of PCs equipped with Intel Xeon X3350 processors with 2667 MHz and 8 Gigabyte of memory. In the following, we first describe the set of benchmark instances that have been used to test our approach. Then, we report on the tuning process that has been employed for finding well-working settings for the three algorithm parameters. Finally, a comparison of the performance of PBIG against the performance of recently published algorithms such as ACO [9], RGES [11] and ACO-SEED [10], is provided.

Table 1
Results for instances of class SPI (Type I).

<i>n</i>	<i>m</i>	OPT	ACO		ACO+SEE		PBIG			Stat. sign.
			Avg	Time (s)	Avg	Evals	Avg	Time (s)	Evals	
10	10	284.0	284.0	0.000	284.0	10	284.0	0.000	4.1	⊖
	20	398.7	398.7	0.008	398.7	18	398.7	0.000	17.5	⊖
	30	431.3	431.3	0.003	431.3	12	431.3	0.000	2.7	⊖
	40	508.5	508.5	0.003	508.5	12	508.5	0.000	1.4	⊖
15	20	441.9	441.9	0.005	441.9	147	441.9	0.000	1.3	⊖
	40	570.4	574.2	0.011	570.4	230	570.4	0.000	2.5	⊖
	60	726.2	729.0	0.008	726.2	45	726.2	0.000	2.8	⊖
	80	807.5	814.6	0.010	807.5	25	807.5	0.000	3.2	⊖
	100	880.0	880.0	0.008	880.0	14	880.0	0.000	3.1	⊖
20	20	473.0	473.0	0.005	473.0	30	473.0	0.000	3.3	⊖
	40	659.3	661.4	0.016	660.3	435	659.3	0.000	22.9	⊖
	60	861.8	861.8	0.014	861.8	189	861.8	0.000	7.7	⊖
	80	898.8 [*]	905.4	0.016	899.9	13	898.0	0.000	17.1	⊖
	100	1026.2	1026.8	0.016	1026.2	538	1026.2	0.000	2.8	⊖
	120	1038.2	1041.5	0.017	1038.2	1011	1038.2	0.000	5.2	⊖
25	40	756.6	756.6	0.019	756.6	103	756.6	0.000	2.1	⊖
	80	1008.1	1009.6	0.022	1008.1	638	1008.1	0.000	12.9	⊖
	100	1106.9	1107.4	0.025	1109.1	1155	1106.9	0.000	12.0	⊖
	150	1264.0	1264.0	0.031	1264.0	50	1264.0	0.000	21.0	⊖
	200	1373.4	1377.7	0.030	1373.4	84	1373.4	0.000	2.7	⊖
AVG			777.4	0.013	776.0	238.0	775.7	0.000	7.42	

^{*} We believe that 898.8 is a mistake from [9]. The 10 solutions of PBIG (resulting in a value of 898.0) were thoroughly checked and are given in Appendix A.

4.1. Problem instances

The algorithm was tested on a set of benchmark instances originally proposed in [9]. Each instance consists of an undirected, vertex-weighted graph with n vertices and m edges. These instances are grouped – with respect to their number of vertices n – into three different classes:

1. **Class SPI:** the class of *small problem instances* (SPI) contains 400 instances. The number of nodes (n) of instances from this class takes values from {10, 15, 20, 25}.
2. **Class MPI:** the class of *moderate-size problem instances* (MPI) consists of 710 instances where n takes values from {50, 100, 150, 200, 250, 300}.

3. **Class LPI:** the class of *large problem instances* (LPI) consists of 15 instances with n from {500, 800, 1000}.

In each class – and for each n – a whole range of instances exists, including rather sparse graphs as well as rather dense graphs. The instances of the first two classes share the following characteristics: (i) 10 problem instances were randomly generated per combination of n and m and results are presented as an average over the objective function values obtained for the 10 instances. (ii) The weight $\omega(v)$ of each vertex $v \in V$ is randomly drawn from a uniform distribution, either from the interval [20, 120] (referred to as Type I) or from the interval $[1, d(v)^2]$ (referred to as Type II), where $d(v)$ is the degree of vertex v . In contrast, class LPI only consists of one problem instance per combination of n and m and the average quality over

Table 2
Results for instances of class SPI (Type II).

<i>n</i>	<i>m</i>	OPT	ACO		ACO+SEE		PBIG			Stat. sign.
			Avg	Time (s)	Avg	Evals	Avg	Time (s)	Evals	
10	10	18.8	18.8	0.003	18.8	45	18.8	0.000	2.4	⊖
	20	51.1	51.1	0.003	51.1	118	51.1	0.000	2.6	⊖
	30	127.9	127.9	0.003	127.9	10	127.9	0.000	1.7	⊖
	40	268.3	268.3	0.010	268.3	20	268.3	0.000	2.0	⊖
15	20	34.7	34.7	0.005	34.7	108	34.7	0.000	14.0	⊖
	40	170.5	171.5	0.010	170.5	716	170.5	0.000	3.3	⊖
	60	360.5	360.8	0.008	360.5	27	360.5	0.000	1.5	⊖
	80	697.9	698.7	0.014	697.9	1663	697.9	0.000	3.9	⊖
	100	1130.4	1137.8	0.008	1130.4	1292	1130.4	0.001	62.4	⊖
20	20	32.9	33.0	0.011	32.9	997	32.9	0.000	1.9	⊖
	40	111.6	111.8	0.017	111.8	1337	111.6	0.000	30.6	⊖
	60	254.1	254.4	0.016	254.1	265	254.1	0.000	12.3	⊖
	80	452.2	453.1	0.016	452.3	235	452.2	0.000	9.1	⊖
	100	775.2	775.2	0.016	775.2	50	775.2	0.000	2.7	⊖
	120	1123.1	1125.5	0.017	1123.1	456	1123.1	0.001	23.4	⊖
25	40	98.7	98.8	0.025	98.7	313	98.7	0.000	30.8	⊖
	80	372.7	373.3	0.026	373.0	238	372.7	0.000	19.6	⊖
	100	595.0	595.1	0.028	595.1	443	595.0	0.000	4.2	⊖
	150	1289.9	1291.7	0.030	1290.9	106	1289.9	0.000	4.8	⊖
	200	2709.5	2713.1	0.030	2709.5	1850	2709.5	0.000	7.9	⊖
AVG			534.7	0.015	533.8	514.5	533.8	0.000	12.1	

Table 3
Results for instances of class MPI (Type I).

<i>n</i>	<i>m</i>	ACO		ACO+SEE		PBIG			Stat. sign.
		Avg	Time (s)	Avg	Evals	Avg	Time (s)	Evals	
50	50	1282.1	0.063	1280.9	414	1280.0	0.016	1152.0	⊖
	100	1741.1	0.083	1740.7	2258	1735.3	0.007	205.9	⊖
	250	2287.4	0.097	2280.6	325	2272.3	0.006	66.4	⊖
	500	2679.0	0.102	2669.3	331	2661.9	0.003	29.9	⊖
	750	2959.0	0.125	2957.3	329	2951.0	0.027	61.1	⊖
	1000	3211.2	0.117	3199.8	2291	3193.7	0.019	24.4	⊖
100	100	2552.9	0.273	2544.0	3722	2537.6	0.019	502.8	⊖
	250	3626.4	0.367	3614.9	1049	3602.7	0.057	428.5	⊖
	500	4692.1	0.433	4636.4	1577	4600.6	0.182	703.0	⊕
	750	5076.4	0.502	5082.8	1454	5045.5	0.088	253.4	⊕
	1000	5534.1	0.456	5522.7	1808	5509.4	0.084	156.3	⊖
	2000	6095.7	0.589	6068.3	448	6051.9	0.501	307.0	⊖
150	150	3684.9	0.691	3676.8	3834	3667.3	0.088	1000.5	⊕
	250	4769.7	0.891	4754.9	2714	4720.3	0.116	713.6	⊕
	500	6224.0	1.194	6228.7	5862	6165.7	0.294	869.4	⊕
	750	7014.7	1.042	6996.3	1705	6963.7	0.522	1348.4	⊕
	1000	7441.8	1.206	7383.6	2090	7368.8	0.536	930.7	⊕
	2000	8631.2	1.103	8597.2	1441	8562.0	0.824	400.3	⊖
	3000	8950.2	0.966	8940.2	584	8899.8	1.300	346.4	⊕
200	250	5588.7	1.674	5572.4	5335	5551.9	0.157	812.2	⊖
	500	7259.2	2.160	7233.7	3930	7192.4	0.547	1374.4	⊕
	750	8349.8	2.602	8300.3	3651	8274.5	0.664	995.6	⊕
	1000	9262.2	2.221	9208.4	3178	9150.6	1.019	1084.6	⊕
	2000	10916.5	2.437	10891.1	2770	10831.0	2.726	1629.2	⊕
	3000	11689.1	2.497	11680.4	1937	11600.2	2.866	690.6	⊕
250	250	6197.8	2.273	6169.2	8418	6148.7	0.392	1956.0	⊕
	500	8538.8	4.016	8495.9	8871	8440.7	1.334	4114.7	⊕
	750	9869.4	4.047	9815.5	8428	9752.8	2.447	4423.5	⊕
	1000	10866.6	3.755	10791.0	6190	10753.7	2.371	2138.9	⊕
	2000	12917.7	3.942	12827.0	6229	12757.6	3.471	1229.8	⊕
	3000	13882.5	4.276	13830.6	5023	13723.5	3.233	896.9	⊕
	5000	14801.8	3.842	14735.9	1507	14676.7	22.508	3004.5	⊕
300	300	7342.7	4.322	7326.6	7370	7296.0	0.711	2557.3	⊕
	500	9517.4	5.178	9491.9	7273	9403.1	1.596	4051.3	⊕
	750	11166.9	6.055	11156.5	8729	11038.1	3.349	3816.9	⊕
	1000	12241.7	6.231	12163.7	9922	12108.9	3.095	2325.1	⊕
	2000	14894.9	6.488	14834.6	4976	14749.9	10.982	3366.3	⊕
	3000	16054.1	6.299	15910.5	3284	15848.2	11.636	2821.3	⊕
AVG	5000	17545.4	6.558	17479.8	3584	17350.6	17.283	1450.5	⊕
		7881.0	2.338	7848.5	3713.9	7806.1	2.489	1390.8	

10 independent runs are generally reported as quality measure of an algorithm. The vertex weights are generated similarly as in the case of the Type I instances.

4.2. Algorithm tuning

As outlined in Section 3, our approach has three parameters for which appropriate values must be determined. This concerns (1) the population size (*pop_size*), (2) the degree of destruction ($p \in (0, 1]$), and (3) the determinism rate ($\alpha \in (0, 1]$). The following values were considered for tuning:

- *pop_size* $\in \{1, 10, 20, 50, 100\}$;
- $p \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$;
- $\alpha \in \{0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$, and

All 150 possible combinations of parameter values were considered, that is, a full factorial design approach was applied. For the tuning experiments we selected 406 problem instances for the training set. More precisely, from each instance class we chose for each value of *n* the instances belonging to the smallest and the largest *m*-value. Then we applied PBIG with each of the 150

parameter value combinations exactly once to each problem instance from the training set. The computation time limit for each run was set to a maximum number of solution evaluations *sol_{max}*. As in previous works (see, for example, [10]) *sol_{max}* was set to 20 000. For example, in case *pop_size* = 50 the maximum number of iterations for PBIG, denoted by *it_{max}*, was set to 400.

For analyzing the tuning experiments, the rank-based procedure as described in [12] has been utilized. For each of the 406 problem instances the 150 different parameter settings were ordered based on the solution values they produced for the corresponding instance. In other words, the combination that found the best solution for a certain instance obtained rank 1, and so on. In the case of ties, the same rank was given. Based on these ranks, an average rank for each parameter combination was computed. The combination with the best average rank for the Type I instances of classes SPI and MPI was (*pop_size* = 100, $p = 0.4$, $\alpha = 0.5$), whereas the combination with the best average rank concerning the Type II instances of classes SPI and MPI was (*pop_size* = 100, $p = 0.5$, $\alpha = 0.8$). Finally, the combination with the best average rank concerning the instances of class LPI was (*pop_size* = 100, $p = 0.3$, $\alpha = 0.5$). Therefore, these parameter settings were adopted (for the corresponding instance classes) for all further experiments.

Table 4
Results for instances of class MPI (Type II).

<i>n</i>	<i>m</i>	ACO		RGES		ACO+SEE		PBIG			Stat. sign.
		Avg	Time (s)	Avg		Avg	Evals	Avg	Time (s)	Evals	
50	50	83.9	0.072	83.9		83.9	1299	83.7	0.002	109.7	⊖
	100	276.2	0.097	276.2		274.4	2716	271.2	0.003	81.3	⊖
	250	1886.8	0.111	1886.4		1870.3	2528	1853.4	0.010	81.1	⊖
	500	7915.9	0.120	7914.5		7876.7	914	7825.1	0.009	45.1	⊕
	750	20134.1	0.111	20134.1		20087.6	505	20079.0	0.010	23.0	⊖
100	50	67.4	0.184	67.4		67.2	266	67.2	0.002	66.0	⊖
	100	169.1	0.334	169.1		167.8	2244	166.6	0.017	199.6	⊖
	250	901.7	0.514	890.4		895.3	6630	886.5	0.065	377.8	⊕
	500	3726.7	0.481	3725.3		3707.0	3806	3693.6	0.101	213.3	⊕
	750	8754.5	0.444	8745.5		8742.3	3000	8680.2	0.129	235.4	⊕
150	50	65.8	0.292	65.8		65.9	190	65.8	0.001	28.1	⊖
	100	144.7	0.583	144.7		144.1	1491	144.0	0.026	231.3	⊖
	250	625.7	1.387	624.4		624.8	6546	616.0	0.187	601.4	⊕
	500	2375.0	1.908	2365.2		2358.6	5925	2331.5	0.572	850.6	⊕
	750	5799.2	1.295	5798.6		5707.0	4928	5698.7	0.550	518.0	⊕
200	50	59.6	0.463	59.6		59.6	48	59.6	0.001	26.6	⊖
	100	134.7	0.981	132.6		134.6	3634	134.5	0.021	188.9	⊖
	250	488.7	2.413	488.4		487.9	7303	483.1	0.280	1054.5	⊕
	500	1843.6	3.423	1843.6		1818.7	10850	1804.3	1.286	1524.8	⊕
	750	4112.8	3.600	4112.8		4077.0	8833	4043.6	0.768	766.0	⊕
250	250	423.2	3.311	423.2		421.2	8553	419.0	0.476	1127.4	⊕
	500	1457.4	5.781	1457.4		1454.3	12500	1435.7	4.219	4113.3	⊕
	750	3315.9	5.983	3315.9		3289.4	7865	3261.0	2.935	1693.0	⊕
	1000	6058.2	6.297	6058.2		6040.0	11586	5989.4	7.978	3223.0	⊕
	2000	26149.1	4.859	26149.1		25932.1	5145	25658.5	11.809	1967.0	⊕
300	5000	171917.2	4.856	171917.2		171500.7	3402	170269.1	37.770	1624.1	⊕
	250	403.9	5.372	403.9		402.7	7713	399.5	0.353	1109.1	⊕
	500	1239.1	9.155	1239.1		1237.3	11971	1216.4	5.439	4510.6	⊕
	750	2678.2	10.994	2678.2		2674.1	9491	2639.4	6.545	4135.8	⊕
	1000	4895.5	9.045	4895.5		4867.9	11492	4796.3	15.204	4941.9	⊕
300	2000	21295.2	7.242	21295.2		21107.7	11399	20891.6	10.911	2084.1	⊕
	5000	143243.5	6.553	143243.5		142292.6	3388	141265.3	27.674	1080.4	⊕
AVG		13832.6	3.071	13831.4		13764.7	5567.5	13663.4	4.230	1213.5	

In general, the tuning results allowed us to draw the following conclusions: as expected, PBIG performs consistently better when adopting a population of solutions rather than a single solution. In fact, in all cases the best-performing parameter combination used a population size of 100. Second, the degree of destruction should rather drop with growing instance sizes, that is, rather small instances seem to require a higher degree of destruction than larger problem instances. Finally, instances of Type II generally require a stronger degree of determinism during the construction of solutions.

4.3. Results

The aim of this section is to compare the performance of PBIG with the best available algorithms from the literature. More precisely, for the comparison we chose the ACO algorithm from [9], the gravity-inspired metaheuristic (referred to as RGES) from [11], and the improved ACO approach (labelled ACO+SEE) from [10]. While ACO and ACO+SEE were applied to all problem instances described in Section 4.1, RGES was only applied to the instances of Type II from class MPI. ACO+SEE can currently be regarded as a state-of-the-art method. In order to be comparable to ACO and ACO+SEE, which were applied for $it_{max} = 2000$ iterations with 10 solution constructions per iteration, we equally used a computation time limit of $sol_{max} = 20\,000$ solution evaluations for each run of PBIG. Moreover, just like ACO and ACO+SEE, PBIG was applied exactly once to each problem instance from classes SPI and MPI, and 10 times to each problem instance from class LPI.

4.3.1. Results concerning class SPI

Tables 1 and 2 present the numerical results obtained by ACO, ACO+SEE and PBIG for the instances of class SPI. Hereby, Table 1 contains the results for the instances of Type I, whereas Table 2 contains the results concerning the Type II instances. Remember that class SPI contains 10 different problem instances for each combination of n and m . The values shown in the tables represent the average results obtained for 10 instances. In general, the structure of Tables 1 and 2 is as follows. The first two columns indicate the number of vertices (n) and the number of edges (m). Column **OPT** refers to the optimal solution values (averaged over 10 instances) as originally provided in [9]. The following two columns provide the average solution quality and the average computation time (in seconds) of ACO. This is the information as given in [9]. The next two columns provide the average solution quality and the average number of solution evaluations of ACO+SEE. Note that the only information provided in [10] is the average solution quality of ACO+SEE. We derived the average number of solution evaluations from additional information provided to us by the authors of [10]. The results of PBIG are provided in three columns. The first one contains the average solution quality, the second one shows the average computation time (in seconds), and the third one provides the average number of solution evaluations. In the case of PBIG we provide both the average computation time and the average number of solution evaluations in order to be able to compare to both ACO and ACO+SEE. Finally, the last table column gives information about the outcome of a statistical test. More specifically, for each combination of n and m we applied the Mann-Whitney test to the results of PBIG and ACO+SEE, which is a current state-of-the-art

Table 5
Results for instances of class LPI.

<i>n</i>	<i>m</i>	ACO+SEE			PBIG				Stat. sign.
		Best	Avg	Evals	Best	Avg	Time (s)	Evals	
500	500	12675	12687.7	10935	12616	12620.0	1.601	3962.8	⊕
	1000	16516	16574.9	10794	16465	16470.1	10.240	12661.1	⊕
	2000	21000	21093.0	7700	20863	20870.8	9.283	4855.1	⊕
	5000	27294	27585.5	6143	27318	27428.2	34.707	6656.1	⊕
	10000	29573	29796.4	6602	29573	29666.8	36.405	2620.4	⊕
800	500	15049	15069.9	4282	15025	15025.0	2.535	5275.9	⊕
	1000	22792	22852.1	10860	22747	22763.0	11.589	9172.0	⊕
	2000	31680	31786.9	8397	31355	31422.6	58.008	17795.6	⊕
	5000	38830	38906.7	11485	38665	38718.7	113.842	12737.6	⊕
	10000	44499	44691.7	6747	44396	44397.8	96.467	4521.9	⊕
1000	1000	24856	24925.4	16377	24746	24763.1	14.435	10034.7	⊕
	5000	45446	45588.7	10225	45255	45295.4	178.311	14860.3	⊕
	10000	51875	52105.0	10298	51378	51540.9	325.956	12113.2	⊕
	15000	58394	58654.8	7277	58014	58145.2	363.179	7906.7	⊕
	20000	60010	60268.2	6896	59790	59847.9	647.563	9291.2	⊕
AVG		33365.9	33505.8	9001.2	33213.7	33265.0	126.941	8964.3	

algorithm. Symbol $\ominus$ indicates that there is no statistical difference between the results of the two algorithms. On the other side, symbol $\oplus$ indicates that the results of PBIG are better than those of ACO+SEE in a statistically significant way (using a standard significance level of 0.05). As a final remark, note that in each table row the result of the best algorithm in the comparison is given in bold font. The last table row provides an average over the whole table.

Concerning the interpretation of the results we can observe that PBIG always generates the optimal result. This can be confirmed by comparing the values in column **OPT** with the average solution qualities obtained by PBIG. In contrast, ACO+SEE is only able to generate the optimal results in 32 out of 40 cases, and ACO is only able to provide the optimal result in 16 out of 40 cases. However, as can be seen in the last column of Tables 1 and 2, there is no statistical difference between the results of ACO+SEE and PBIG. For what concerns computation time, we can observe that PBIG is very fast. In fact, computation times are smaller than 0.001 seconds in nearly all cases. ACO, on the contrary, has average computation times of 0.013 seconds (see the last row of Table 1) and 0.015 seconds (see the last row of Table 2). In order to be able to compare these computation times we must note that ACO has been executed on a computer with an AMD processor of 1700 MHz and 256 MB of RAM. Therefore, we can assume that the machines that we used for executing PBIG are about twice as fast as the one used for executing ACO. When comparing the average number of solution evaluations of ACO+SEE and PBIG, we can observe that PBIG needs between 30 times (Table 1) and 40 times (Table 2) less solution evaluations than ACO+SEE. In summary, the small problem instances of class SPI do not pose any challenge to PBIG.

4.3.2. Results concerning class MPI

The results for the problem instances of class MPI are presented in Tables 3 and 4, in the same way as described in the previous section for the case of instance class SPI. The additional column **RGES** in Table 4 presents the results of the RGES algorithm [11] which was only applied to the Type II instances of class MPI. Note also that optimal solutions are not known for the instances of this class, due to their increased size. Therefore, column **OPT** is missing. Again we can observe that PBIG is superior to the competitor algorithms. In all 70 cases PBIG obtains the best result of the comparison. Altogether, ACO, RGES, and ACO+SEE are only able to match the results of PBIG in two cases. Concerning statistical significance, it can be observed that for rather small problem instances of this class, the

improvement of PBIG over ACO+SEE is still not statistically significant. On the other side, for all larger problem instances, the improvement of PBIG over ACO+SEE is statistically significant. Altogether, statistical significance could be determined in 49 out of 70 cases. For what concerns a comparison of computation time, we can observe that PBIG needs about twice as much computation time than ACO. This is taking into consideration the fact that the machines used to execute PBIG are about twice as fast as the machine used to run ACO. However, given that the stopping criterion for ACO and PBIG has been the same, this only means that ACO gets stuck earlier during the search process, while PBIG is able to make a better use of the allotted computation time. Finally, comparing the number of solution evaluations needed by ACO+SEE and PBIG, it can be observed that PBIG obtains a better solution quality than ACO+SEE using, on average, about 3–4 times fewer solution evaluations.

4.3.3. Results concerning class LPI

Table 5 presents a comparison of PBIG with ACO+SEE for what concerns the instances of class LPI. Remember that the instances of this class are the largest ones of the whole benchmark set. In this case we only compare to ACO+SEE, which is the best available algorithm for these instances. In contrast to instance classes SPI and MPI, class LPI only contains one single instance per combination of *n* and *m*. Therefore, the results are given as averages over 10 runs per instance. For each of the two algorithms, columns **Best**, **Avg** and **Evals** provide, respectively, the value of the best solution found, the average solution quality over 10 runs and the average number of solution evaluations needed for reaching the best solutions in 10 runs. In addition, in the case of PBIG we provide the average computation time in column **Time (s)**. For each combination we compare between the best performance (columns **Best**) and the average performance (columns **Avg**) of PBIG and ACO+SEE. The better among the two is always indicated in bold font. Concerning the comparison, we can observe that PBIG outperforms ACO+SEE both for what concerns the best solutions found and the average solution quality obtained. The only exception is case (*n* = 500, *m* = 5000) where ACO+SEE finds a better solution than PBIG. It is interesting to note that in 13 out of 15 cases the average solution quality obtained by PBIG is better than the value of the best solution found by ACO+SEE. Moreover, the last table column indicates that in all cases PBIG improves over ACO+SEE in a statistical significant way. Concerning the number of solution evaluations, PBIG and ACO+SEE make use of, on average, a comparable number of solution evaluations.

5. Conclusions

We have proposed a population-based iterated greedy (PBIG) approach to tackle the minimum weight vertex cover problem. To the best of our knowledge, this is the first attempt to adopt a population of solutions in the context of iterated greedy algorithms. The proposed algorithm maintains a population of solutions and applies, at each iteration, the basic steps of an iterated greedy algorithm to each member of the population. The best solutions among the current population and the newly generated solutions are accepted for the population of the subsequent iteration.

Tuning experiments have confirmed that working with a population of solutions rather than being restricted to a single solution can greatly avoid the rapid convergence to local optima and increase diversity. The obtained results based on 1125 benchmark instances of different sizes have shown that PBIG significantly outperforms the currently best algorithms from the literature especially when the instance size grows. Furthermore, PBIG is able to solve all small problem instances to optimality, in negligible computation time. In the future, we plan to extend our approach by using several greedy heuristics at the same time, rather than being restricted to a single one.

Appendix A.

Here we report the best solutions found by PBIG for the 10 instances of case ($n=20$, $m=80$) from Table 1. Note that the 20 vertices are numbered from 1 to 20.

Instance no. 1

$S = \{6, 9, 15, 5, 3, 8, 7, 17, 13, 18, 16, 11, 12, 20\}$
 $\omega(S) = 899$

Instance no. 2

$S = \{9, 8, 15, 13, 6, 2, 14, 5, 3, 12, 10, 7, 18, 1, 20, 16\}$
 $\omega(S) = 947$

Instance no. 3

$S = \{14, 10, 4, 16, 19, 8, 7, 18, 17, 5, 20, 12, 11, 2\}$
 $\omega(S) = 866$

Instance no. 4

$S = \{3, 15, 14, 4, 17, 5, 7, 11, 1, 19, 10, 20, 9, 2\}$
 $\omega(S) = 768$

Instance no. 5

$S = \{16, 17, 1, 20, 6, 15, 2, 3, 7, 18, 9, 11, 14, 4\}$
 $\omega(S) = 976$

Instance no. 6

$S = \{20, 13, 7, 1, 14, 15, 16, 2, 9, 12, 10, 11, 5, 3\}$
 $\omega(S) = 905$

Instance no. 7

$S = \{7, 11, 4, 18, 9, 8, 3, 14, 20, 12, 1, 16, 10, 19\}$
 $\omega(S) = 944$

Instance no. 8

$S = \{18, 15, 13, 16, 9, 14, 19, 8, 5, 4, 2, 6, 10, 17, 3\}$
 $\omega(S) = 899$

Instance no. 9

$S = \{15, 9, 4, 18, 3, 11, 20, 5, 1, 14, 10, 6, 2, 13\}$
 $\omega(S) = 970$

Instance no. 10

$S = \{18, 15, 3, 13, 1, 9, 4, 6, 12, 10, 5, 7, 20, 2\}$
 $\omega(S) = 806$

Average solution quality: 898.0

References

- [1] R.M. Karp, Reducibility among combinatorial problems, in: R.E. Miller, J.W. Thatcher (Eds.), *Complexity of Computer Computations*, Plenum Press, New York, 1972, pp. 85–103.
- [2] V. Chvátal, A greedy heuristic for the set-covering problem, *Mathematics of Operations Research* 4 (3) (1979) 233–235.
- [3] L. Pitt, A simple probabilistic approximation algorithm for vertex cover, Tech. Rep. YaleU/DCS/TR-404, Department of Computer Science, Yale University, 1985.
- [4] D.S. Hochbaum, *Approximation Algorithms for NP-Hard Problems*, PWS, Boston, 1997.
- [5] I. Dinur, S. Safra, On the hardness of approximating minimum vertex-cover, *Annals of Mathematics* 162 (2005) 439–485.
- [6] S. Khot, O. Regev, Vertex cover might be hard to approximate to within $2 - \epsilon$, in: 18th Annual IEEE Conference on Computational Complexity (CCC'03), 2003, pp. 379–386.
- [7] S. Khuri, T. Bäck, An evolutionary heuristic for the minimum vertex cover problem, in: J. Hopf (Ed.), *Genetic Algorithms within the Framework of Evolutionary Computation*, KI-94 Workshop, Saarbrücken, Germany, 1994, pp. 86–90.
- [8] A. Singh, A.K. Gupta, A hybrid heuristic for the minimum weight vertex cover problem, *Asia-Pacific Journal of Operational Research* 23 (2) (2006) 273–285.
- [9] S. Shyu, P. Yin, B. Lin, An ant colony optimization algorithm for the minimum weight vertex cover problem, *Annals of Operations Research* 131 (1–4) (2004) 283–304.
- [10] R. Jovanovic, M. Tuba, An ant colony optimization algorithm with improved pheromone correction strategy for the minimum weight vertex cover problem, *Applied Soft Computing Journal* 11 (8) (2011) 5360–5366.
- [11] S.R. Balachandar, K. Kannan, A meta-heuristic algorithm for vertex covering problem based on gravity, *International Journal of Mathematical and Statistical Sciences* 1 (3) (2009) 130–136.
- [12] S. Benedettini, C. Blum, A. Roli, A randomized iterated greedy algorithm for the founder sequence reconstruction problem, in: C. Blum, R. Battiti (Eds.), *Learning and Intelligent Optimization*, 4th International Conference (LION 4), Vol. 607 of *Lecture Notes in Computer Science*, Springer, 2010, pp. 37–51.
- [13] R. Ruiz, T. Stützle, A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem, *European Journal of Operational Research* 177 (3) (2007) 2033–2049.
- [14] L. Fanjul-Peyro, R. Ruiz, Iterated greedy local search methods for unrelated parallel machine scheduling, *European Journal of Operational Research* 207 (1) (2010) 55–69.
- [15] I. Ribas, R. Companys, X. Tort-Martorell, An iterated greedy algorithm for the flowshop scheduling problem with blocking, *Omega* 39 (3) (2011) 293–301.
- [16] M. Lozano, D. Molina, C. García-Martínez, Iterated greedy for the maximum diversity problem, *European Journal of Operational Research* 214 (1) (2011) 31–38.
- [17] H.H. Hoos, T. Stützle, *Stochastic Local Search: Foundations & Applications*, Elsevier / Morgan Kaufmann, San Francisco, CA, USA, 2004.